JAVA COURSE

{ scanner e input }



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.1

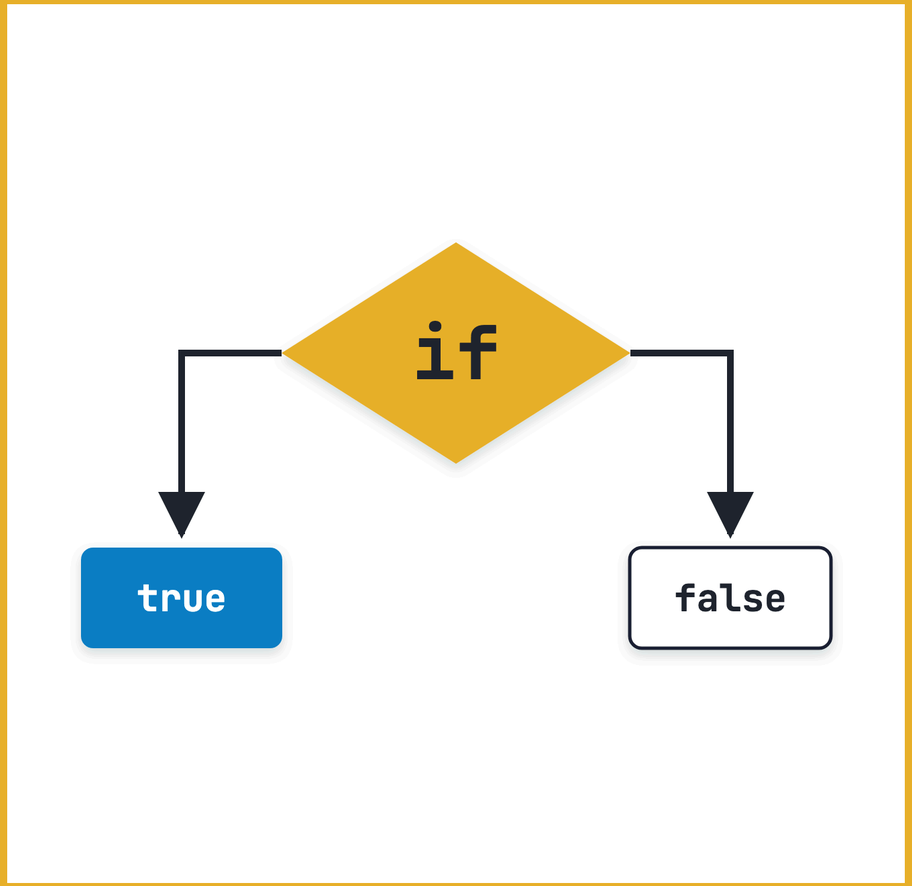
INDICE

01 Leggere l'input

02 Validare l'input

03 Scanner in pratica

01
LEGGERE L'INPUT



PERCHÉ SERVE

Finora ogni dato era scritto direttamente nel codice sorgente. Per creare programmi utili serve **ricevere dati dall'utente** mentre il programma è in esecuzione

- *Chiedere un nome, un numero, una scelta*
- *Il programma **si ferma**, aspetta la risposta e poi prosegue*
- *Java offre la classe* `Scanner` *per leggere l'input da tastiera*

LA CLASSE SCANNER

`Scanner` è una classe della libreria standard che legge testo da una sorgente. Per leggere dalla tastiera si collega a `System.in`

```
import java.util.Scanner;

Scanner scanner = new Scanner(System.in);
```

- `import` *dice al compilatore dove trovare la classe*
- `new Scanner(System.in)` *crea un oggetto che legge dalla tastiera*
- *L'oggetto si conserva in una variabile per usarlo più volte*

NEXTLINE

`nextLine()` legge una riga intera di testo e la restituisce come `String`. Il programma si ferma finché l'utente non preme Invio

```java
import java.util.Scanner;

public class Saluto {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Come ti chiami? ");
        String nome = scanner.nextLine();
        System.out.println("Ciao, " + nome + "!");
    }
}
```

Terminale

```
Come ti chiami? Al
Ciao, Alice!
```

NEXTINT E NEXTDOUBLE

`Scanner` offre metodi tipizzati: leggono il token successivo e lo convertono nel tipo richiesto

Metodo	Tipo restituito	Esempio input
`nextInt()`	`int`	`42`
`nextDouble()`	`double`	`3.14`
`nextBoolean()`	`boolean`	`true`
`nextLine()`	`String`	`Ciao mondo`
`next()`	`String`	`Ciao`

LEGGERE UN NUMERO

`nextInt()` legge un intero dal terminale. Se l'utente scrive qualcosa che non è un numero, il programma lancia un'eccezione

```java
Scanner scanner = new Scanner(System.in);

System.out.print("Quanti anni hai? ");
int eta = scanner.nextInt();

System.out.println("Tra 10 anni ne avrai " + (eta + 10));
```

Terminale

```
Quanti anni hai? 20
Tra 10 anni ne avrai
```

IL PROBLEMA DEL NEWLINE

Quando l'utente preme Invio dopo un numero, il carattere `\n` resta nel buffer. Se dopo `nextInt()` si chiama `nextLine()`, questa trova il `\n` e restituisce una stringa vuota

```
System.out.print("Età: ");
int eta = scanner.nextInt();
// nel buffer resta '\n'

System.out.print("Nome: ");
String nome = scanner.nextLine();  // legge "" (stringa vuota!)
```

- *La soluzione: aggiungere un* `scanner.nextLine()` *subito dopo* `nextInt()` *per consumare il* `\n` *residuo*

LA SOLUZIONE

Si inserisce una chiamata a `nextLine()` subito dopo `nextInt()` o `nextDouble()` per scartare il newline rimasto nel buffer

```
System.out.print("Età: ");
int eta = scanner.nextInt();
scanner.nextLine();                // consuma il '\n' residuo

System.out.print("Nome: ");
String nome = scanner.nextLine(); // ora legge il nome correttamente

System.out.println(nome + " ha " + eta + " anni");
```

Terminale

```
Età: 20
Nome: Alice
Alice ha 20 anni
```

CHIUDERE LO SCANNER

In teoria `scanner.close()` libera le risorse. In pratica chiudere uno Scanner su `System.in` chiude anche lo standard input: nessun altro Scanner potrà più leggere

- *Nei programmi didattici **non serve chiuderlo**: il sistema operativo libera tutto quando il processo termina*
- *Su file o rete la chiusura è importante: si usa* `try-with-resources`

```
// con try-with-resources (file, rete)
try (Scanner sc = new Scanner(new File("dati.txt"))) {
    String riga = sc.nextLine();
}
```

ESEMPIO COMPLETO

Un programma che legge nome, cognome, età e li stampa in una frase

```java
import java.util.Scanner;

public class SchedaPersonale {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Nome: ");
        String nome = scanner.nextLine();

        System.out.print("Cognome: ");
        String cognome = scanner.nextLine();

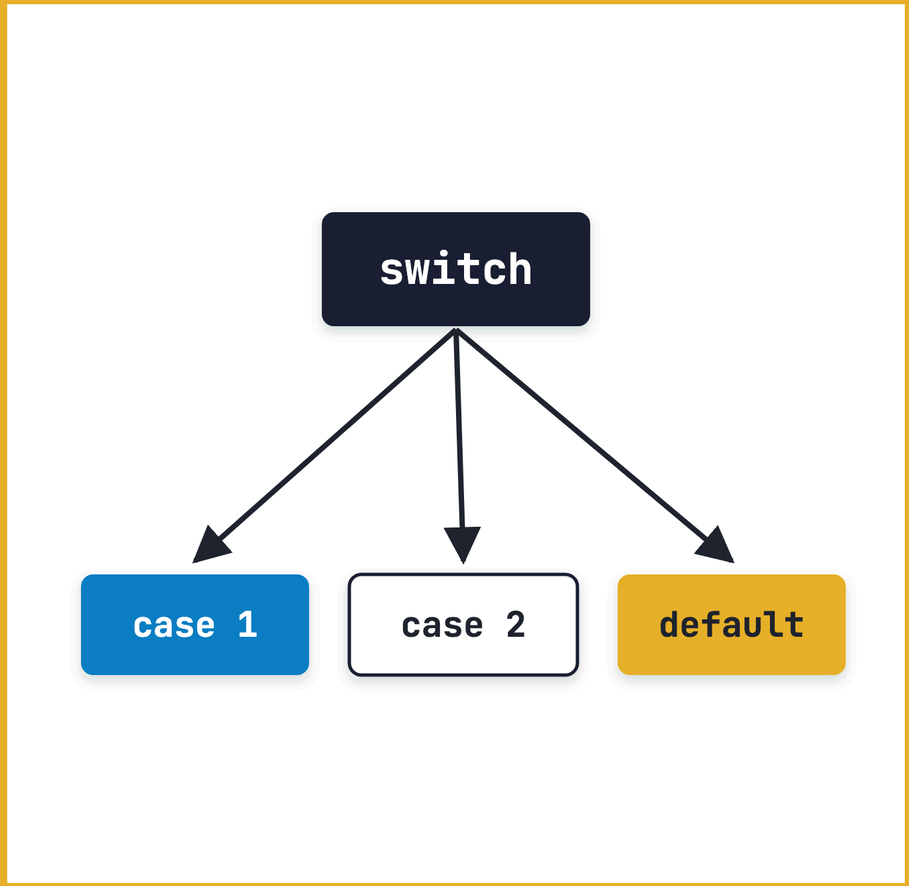
        System.out.print("Età: ");
        int eta = scanner.nextInt();

        System.out.println(nome + " " + cognome + ", " + eta + " anni");
    }
}
```

Terminale

```
Nome: Alice
Cognome: Rossi
Età: 22
Alice Rossi, 22 anni
```

02
VALIDARE L'INPUT



INPUTMISMATCHEXCEPTION

Se l'utente scrive `abc` quando il programma si aspetta un intero, `nextInt()` lancia `InputMismatchException` e il programma si interrompe

```
System.out.print("Inserisci un numero: ");
int n = scanner.nextInt();   // l'utente scrive "abc"
```

Terminale

```
Inserisci un numero: abc
Exception in thread "main"
  java.util.InputMismatchException
```

- *Per evitarlo bisogna **controllare l'input prima di leggerlo***

HASNEXTINT E HASNEXTDOUBLE

I metodi `hasNextInt()` e `hasNextDouble()` controllano se il prossimo token nel buffer è del tipo atteso, **senza consumarlo**

```
System.out.print("Inserisci un numero: ");

if (scanner.hasNextInt()) {
    int n = scanner.nextInt();
    System.out.println("Hai inserito: " + n);
} else {
    System.out.println("Input non valido");
    scanner.next();   // scarta il token errato
}
```

IL CICLO DI VALIDAZIONE

Un `while` che continua a chiedere finché l'utente non inserisce un dato valido

```
System.out.print("Inserisci un intero: ");

while (!scanner.hasNextInt()) {
    System.out.println("Non è un intero, riprova");
    scanner.next();   // scarta il token non valido
    System.out.print("Inserisci un intero: ");
}

int numero = scanner.nextInt();
System.out.println("Hai inserito: " + numero);
```

VALIDARE UN INTERVALLO

Oltre al tipo si può controllare che il valore rientri in un intervallo. Il ciclo combina `hasNextInt()` con un controllo sul valore letto

```
int voto;

while (true) {
    System.out.print("Voto (1-10): ");
    if (!scanner.hasNextInt()) {
        System.out.println("Inserisci un numero");
        scanner.next();
        continue;
    }
    voto = scanner.nextInt();
    if (voto >= 1 && voto <= 10) {
        break;
    }
    System.out.println("Il voto deve essere tra 1 e 10");
}

System.out.println("Voto registrato: " + voto);
```

PARSEINT E NUMBERFORMATEXCEPTION

Un approccio alternativo: leggere sempre una `String` con `nextLine()` e convertirla con `Integer.parseInt()`. Se la stringa non è un numero, viene lanciata `NumberFormatException`

```
String input = scanner.nextLine();

int numero = Integer.parseInt(input);     // "42" → 42
double decimale = Double.parseDouble(input); // "3.14" → 3.14
```

- *Vantaggio: nessun problema di newline residuo*
- *Svantaggio: serve gestire l'eccezione (lo vedremo più avanti)*

NEXT VS NEXTLINE

`next()` legge un singolo token (fino al primo spazio). `nextLine()` legge tutta la riga

```
// L'utente scrive "Ciao mondo"

String parola = scanner.next();       // "Ciao"
String resto  = scanner.nextLine();   // " mondo"
```

Metodo	Legge fino a	Consuma il newline
`next()`	primo spazio	no
`nextLine()`	fine riga	si

CHE COSA STAMPA?

L'utente scrive `25` e poi `Alice`. Che cosa stampa il programma?

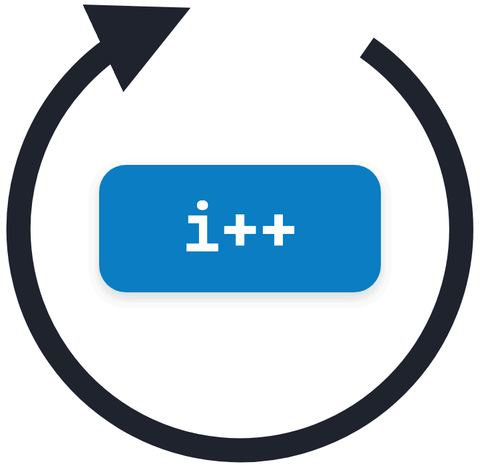
```
Scanner scanner = new Scanner(System.in);

System.out.print("Età: ");
int eta = scanner.nextInt();

System.out.print("Nome: ");
String nome = scanner.nextLine();

System.out.println("[" + nome + "] ha " + eta + " anni");
// ??
```

03
SCANNER IN PRATICA



MENU TESTUALE

Un `do-while` che mostra le opzioni, legge la scelta e usa uno `switch` per eseguire l'azione

```
Scanner scanner = new Scanner(System.in);
int scelta;

do {
    System.out.println("1. Saluta");
    System.out.println("2. Calcola");
    System.out.println("0. Esci");
    System.out.print("Scelta: ");
    scelta = scanner.nextInt();

    switch (scelta) {
        case 1 -> System.out.println("Ciao!");
        case 2 -> System.out.println("2 + 2 = " + (2 + 2));
        case 0 -> System.out.println("Arrivederci");
        default -> System.out.println("Scelta non valida");
    }
} while (scelta != 0);
```

LEGGERE PIÙ VALORI IN UN CICLO

Un ciclo che legge numeri finché l'utente non inserisce un valore sentinella.

Al termine stampa la somma

```
Scanner scanner = new Scanner(System.in);
int somma = 0;

while (true) {
    System.out.print("Numero (0 per terminare): ");
    if (!scanner.hasNextInt()) {
        System.out.println("Non è un numero, riprova");
        scanner.next();
        continue;
    }
    int n = scanner.nextInt();
    if (n == 0) break;
    somma += n;
}

System.out.println("Somma: " + somma);
```

PRINT VS PRINTLN

Per le domande all'utente si usa `print` (senza `ln`): il cursore resta sulla stessa riga e l'utente scrive accanto alla domanda

```
// con println: il cursore va a capo
System.out.println("Nome: ");
// Nome:
// _                ← l'utente scrive qui

// con print: il cursore resta sulla riga
System.out.print("Nome: ");
// Nome: _          ← l'utente scrive qui
```

LOCALE E SEPARATORE DECIMALE

`nextDouble()` usa il separatore decimale del sistema: in Italia la virgola, negli USA il punto. Per forzare il punto si imposta il `Locale`

```
import java.util.Locale;
import java.util.Scanner;

Scanner scanner = new Scanner(System.in);
scanner.useLocale(Locale.US);    // forza il punto come separatore

System.out.print("Prezzo: ");
double prezzo = scanner.nextDouble();   // 9.99 funziona
```

- *Senza `useLocale`, su un sistema italiano bisogna scrivere `9,99`*
- *In alternativa: leggere con `nextLine()` e convertire con `Double.parseDouble()`, che usa sempre il punto*

CONFRONTO CON PYTHON

In Python `input()` restituisce sempre una stringa; la conversione è esplicita. In Java `Scanner` offre metodi tipizzati ma richiede più codice

Aspetto	Python	Java
Leggere una stringa	`input()`	`scanner.nextLine()`
Leggere un intero	`int(input())`	`scanner.nextInt()`
Newline residuo	non esiste	da gestire manualmente
Validazione tipo	`try/except`	`hasNextInt()`
Prompt	dentro `input()`	con `print()` separato
Import	nessuno	`import java.util.Scanner`

SCHEMA: LEGGERE UN VALORE

I tre passaggi per leggere un valore dall'utente

- ***1. Stampare la domanda*** *con* `System.out.print()`
- ***2. Leggere la risposta*** *con il metodo appropriato di* `Scanner`
- ***3. Consumare il newline*** *se si è usato* `nextInt()` *o* `nextDouble()` *e dopo serve* `nextLine()`

```
System.out.print("Età: ");       // 1. domanda
int eta = scanner.nextInt();    // 2. lettura
scanner.nextLine();             // 3. newline
```

ERRORI COMUNI

I problemi che si incontrano più spesso con `Scanner`

Errore	Conseguenza
Dimenticare `import java.util.Scanner`	`cannot find symbol`
Non consumare il `\n` dopo `nextInt()`	`nextLine()` restituisce stringa vuota
Usare `nextInt()` con input non numerico	`InputMismatchException`
Chiudere lo Scanner e riaprirne un altro	`NoSuchElementException`
Usare `==` per confrontare la stringa letta	Confronto di riferimenti, non di contenuto
Virgola come separatore con Locale italiano	`InputMismatchException` con il punto

CHE COSA SUCCEDE?

L'utente inserisce `ciao` quando il programma chiede un numero. Che cosa succede?

```
Scanner scanner = new Scanner(System.in);

System.out.print("Numero: ");

while (!scanner.hasNextInt()) {
    System.out.println("Non valido, riprova");
    scanner.next();
    System.out.print("Numero: ");
}

int n = scanner.nextInt();
System.out.println("Letto: " + n);
// ??
```

DOMANDE?



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.1